

EngiSense AI

10-Phase Development Roadmap

AI-Powered Engineering Intelligence Platform

React + FastAPI + PostgreSQL
LangChain + Hugging Face + RAG
Pandas + NumPy + Matplotlib + Seaborn
Docker + Git + GitHub + CI/CD

Document Type: Master Development Roadmap
Project Type: Full-Stack AI / Data Analytics
Development Model: Incremental / Phase-Based
Number of Phases: 10
Version: 1.0

Development Reference Document

Contents

1	Purpose of This Document	4
2	Overall Development Flow	4
3	Master Phase Overview	5
4	Phase 1 – Product Planning and System Architecture	6
4.1	Objective	6
4.2	Tools	6
4.3	Tasks	6
4.4	Initial MVP	7
4.5	Deliverables	7
4.6	Definition of Done	8
5	Phase 2 – FastAPI Backend and PostgreSQL Foundation	8
5.1	Objective	8
5.2	Tools	8
5.3	Tasks	9
5.4	Initial Database Entities	9
5.5	Initial Backend Structure	9
5.6	Deliverables	10
5.7	Definition of Done	10
6	Phase 3 – REST APIs and Authentication	10
6.1	Objective	10
6.2	Tools	10
6.3	Authentication APIs	11
6.4	Project APIs	11
6.5	Dataset APIs	11
6.6	Tasks	11
6.7	Deliverables	12
6.8	Definition of Done	12
7	Phase 4 – Engineering Data Analytics Engine	12
7.1	Objective	12
7.2	Tools	12
7.3	Tasks	13
7.4	Statistical Operations	13
7.5	Visualization Operations	13
7.6	Example Analysis Pipeline	14
7.7	Deliverables	14
7.8	Definition of Done	14

8	Phase 5 – React Frontend and Full-Stack Integration	15
8.1	Objective	15
8.2	Tools	15
8.3	Initial Pages	15
8.4	Tasks	15
8.5	Full-Stack Flow	16
8.6	Definition of Done	16
9	Phase 6 – Document Intelligence and RAG	16
9.1	Objective	16
9.2	Tools	17
9.3	Tasks	17
9.4	RAG Pipeline	17
9.5	Definition of Done	18
10	Phase 7 – GenAI and Data-Aware AI	18
10.1	Objective	18
10.2	Tools	18
10.3	AI Tools	19
10.4	AI Architecture	19
10.5	Example	19
10.6	Security Principle	20
10.7	Definition of Done	20
11	Phase 8 – Docker and Production Engineering	20
11.1	Objective	20
11.2	Tools	20
11.3	Services	21
11.4	Docker Architecture	21
11.5	Tasks	21
11.6	Definition of Done	22
12	Phase 9 – Testing, Security and CI/CD	22
12.1	Objective	22
12.2	Tools	22
12.3	Backend Testing	22
12.4	Data Testing	23
12.5	Security Testing	23
12.6	CI Pipeline	23
12.7	Definition of Done	24
13	Phase 10 – Deployment and Final Product Release	24
13.1	Objective	24
13.2	Tools	24
13.3	Tasks	24
13.4	Final Documentation	25
13.5	Portfolio Materials	25
13.6	Definition of Done	25

14 Standard Development Workflow	26
15 Git and GitHub Strategy	27
16 Target Project Structure	27
17 Global Definition of Done	28
18 Master Progress Checklist	28
19 Expected Final Architecture	29
20 Final Project Goal	30

1 Purpose of This Document

This document defines the complete development roadmap for the EngiSense AI project.

The purpose of this roadmap is to provide a structured development path so that the project can be implemented systematically rather than developing unrelated features independently.

The project is divided into ten major phases.

Each phase contains:

- Objective
- Technologies and tools
- Development tasks
- Expected deliverables
- Definition of Done

The phases are ordered according to their technical dependencies.

The development process should follow the sequence defined in this document unless an architectural decision requires modification.

2 Overall Development Flow

The complete development process follows:

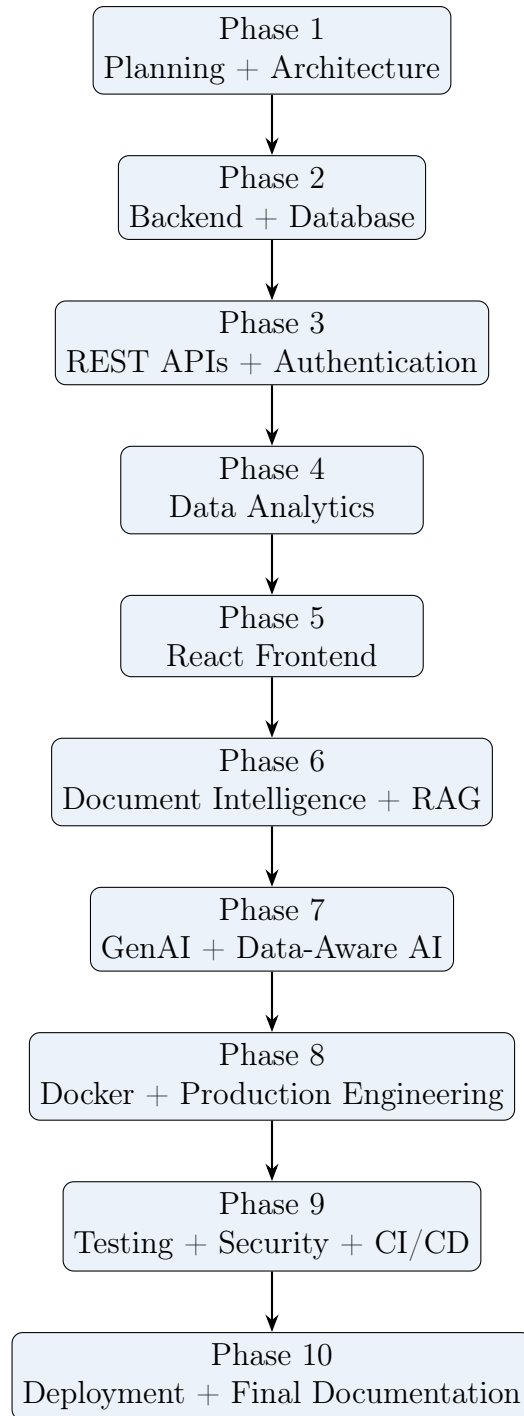


Figure 1: EngiSense AI Development Flow

3 Master Phase Overview

No.	Phase	Main Technologies	Primary Output
1	Planning and Architecture	GitHub, Markdown, Draw.io	Requirements, architecture and backlog

2	Backend Foundation and Database	FastAPI, PostgreSQL, SQLAlchemy, Alembic	Working backend and database
3	REST APIs and Authentication	FastAPI, Pydantic, JWT, Postman	Complete API layer and authentication
4	Data Analytics Engine	Pandas, NumPy, Matplotlib, Seaborn	Engineering analytics engine
5	React Frontend	React, Vite, Tailwind CSS, Axios	Functional full-stack UI
6	Document Intelligence and RAG	LangChain, Hugging Face, Embeddings, Vector Store	Document-based AI retrieval
7	GenAI and Data-Aware AI	LangChain, Hugging Face, Pandas, RAG	AI assistant with tools
8	Docker and Production Engineering	Docker, Docker Compose, Nginx	Containerized application
9	Testing, Security and CI/CD	Pytest, GitHub Actions, ESLint, Ruff	Validated and automated project
10	Deployment and Final Release	Docker, Cloud, GitHub	Deployed and documented product

4 Phase 1 – Product Planning and System Architecture

4.1 Objective

Define exactly what EngiSense AI will build before implementation begins.

This phase establishes the product scope, technical architecture, requirements and development backlog.

4.2 Tools

- Git
- GitHub
- VS Code
- Markdown
- Draw.io / Excalidraw
- Overleaf

4.3 Tasks

1. Finalize the project problem statement.
2. Define target users.
3. Define user personas.

4. Define major use cases.
5. Define MVP scope.
6. Define functional requirements.
7. Define non-functional requirements.
8. Define technology stack.
9. Design high-level system architecture.
10. Define major application modules.
11. Create initial GitHub repository.
12. Create project backlog.
13. Define Definition of Done.

4.4 Initial MVP

The initial MVP should contain:

```
User Authentication
|
v
Project Management
|
+----- Dataset Upload
|
+----- Dataset Analysis
|
+----- Visualization
|
+----- Document Upload
|
+----- RAG
|
+----- AI Chat
```

4.5 Deliverables

```
docs/
|
+-- requirements.md
+-- architecture.md
+-- use-cases.md
+-- project-roadmap.md
+-- development-backlog.md
```

4.6 Definition of Done

Phase 1 is complete when:

- The problem statement is finalized.
- MVP features are finalized.
- System architecture is documented.
- Technology choices are documented.
- Major database entities are identified.
- API modules are identified.
- Development backlog is created.

5 Phase 2 – FastAPI Backend and PostgreSQL Foundation

5.1 Objective

Build the core backend infrastructure and establish the connection between FastAPI and PostgreSQL.

5.2 Tools

- Python
- FastAPI
- Uvicorn
- PostgreSQL
- SQLAlchemy
- Pydantic
- Alembic
- VS Code
- Git

5.3 Tasks

1. Create backend project.
2. Create Python virtual environment.
3. Install dependencies.
4. Configure FastAPI.
5. Create application entry point.
6. Create health-check endpoint.
7. Install and configure PostgreSQL.
8. Configure database connection.
9. Configure SQLAlchemy.
10. Create database session management.
11. Configure Alembic.
12. Create initial database models.

5.4 Initial Database Entities

```
User
Project
Dataset
Document
AnalysisResult
ChatSession
ChatMessage
```

5.5 Initial Backend Structure

```
backend/
|
+-- app/
| |
| +-- main.py
| |
| +-- core/
| |
| +-- database/
| |
| +-- models/
| |
| +-- schemas/
| |
```

```
| +-- services/  
| |  
| +-- api/  
|  
+-- tests/  
|  
+-- requirements.txt  
+-- .env  
+-- alembic.ini
```

5.6 Deliverables

- Working FastAPI application.
- Working PostgreSQL database.
- SQLAlchemy integration.
- Alembic migration system.
- Initial database models.

5.7 Definition of Done

Phase 2 is complete when:

```
FastAPI  
|  
v  
SQLAlchemy  
|  
v  
PostgreSQL
```

is working correctly and database operations can be performed successfully.

6 Phase 3 – REST APIs and Authentication

6.1 Objective

Build a complete REST API layer and secure the application using authentication and authorization.

6.2 Tools

- FastAPI
- Pydantic
- SQLAlchemy
- PostgreSQL

- JWT
- Password hashing library
- Postman
- OpenAPI / Swagger

6.3 Authentication APIs

```
POST /auth/register  
POST /auth/login  
GET /auth/me
```

6.4 Project APIs

```
POST /projects  
GET /projects  
GET /projects/{id}  
PUT /projects/{id}  
DELETE /projects/{id}
```

6.5 Dataset APIs

```
POST /datasets/upload  
GET /datasets  
GET /datasets/{id}  
DELETE /datasets/{id}
```

6.6 Tasks

1. Implement registration.
2. Implement password hashing.
3. Implement login.
4. Implement JWT authentication.
5. Implement protected routes.
6. Implement project CRUD.
7. Implement dataset metadata APIs.
8. Implement validation using Pydantic.
9. Configure CORS.
10. Test APIs using Postman.
11. Verify Swagger/OpenAPI documentation.

6.7 Deliverables

- Authentication system.
- Project management APIs.
- Dataset management APIs.
- API documentation.
- Postman test collection.

6.8 Definition of Done

A developer should be able to use the Swagger documentation or Postman to:

```
Register
  |
Login
  |
Receive JWT
  |
Create Project
  |
Read Project
  |
Update Project
  |
Delete Project
```

7 Phase 4 – Engineering Data Analytics Engine

7.1 Objective

Create a reusable data-analysis engine that converts uploaded datasets into meaningful engineering information.

7.2 Tools

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- FastAPI

7.3 Tasks

1. Implement dataset ingestion.
2. Validate uploaded CSV files.
3. Load data into Pandas DataFrame.
4. Detect data types.
5. Calculate dataset dimensions.
6. Detect missing values.
7. Detect duplicates.
8. Generate descriptive statistics.
9. Calculate correlations.
10. Implement outlier detection.
11. Generate charts.
12. Store analysis metadata.
13. Expose analysis through APIs.

7.4 Statistical Operations

Mean
Median
Standard Deviation
Minimum
Maximum
Percentiles
Variance
Correlation

7.5 Visualization Operations

Line Plot
Bar Chart
Histogram
Scatter Plot
Box Plot
Correlation Heatmap

7.6 Example Analysis Pipeline

```

CSV
|
v
Pandas DataFrame
|
+---- Data Profiling
|
+---- Statistics
|
+---- Correlation
|
+---- Outlier Detection
|
+---- Visualization
|
v
Analysis Results

```

7.7 Deliverables

- Data processing service.
- Statistical analysis service.
- Visualization service.
- Analysis APIs.

7.8 Definition of Done

The backend must be able to receive a dataset and generate:

```

Dataset Summary
+
Statistics
+
Missing Value Report
+
Correlation Information
+
Outlier Information
+
Charts

```

without requiring an LLM.

8 Phase 5 – React Frontend and Full-Stack Integration

8.1 Objective

Build the user-facing application and connect it to the backend APIs.

8.2 Tools

- React
- Vite
- JavaScript / TypeScript
- Tailwind CSS
- Axios / Fetch
- React Router
- Git

8.3 Initial Pages

```
/login  
/register  
/dashboard  
/projects  
/projects/:id  
/datasets  
/analysis  
/documents  
/chat  
/settings
```

8.4 Tasks

1. Create React application.
2. Configure routing.
3. Create application layout.
4. Create navigation.
5. Create authentication pages.
6. Connect authentication APIs.
7. Implement protected routes.
8. Create dashboard.

9. Create project interface.
10. Create dataset upload interface.
11. Display dataset statistics.
12. Display charts.
13. Connect all required APIs.

8.5 Full-Stack Flow

```
React
|
| HTTP Request
v
FastAPI
|
v
Service Layer
|
+---- PostgreSQL
|
+---- Data Analysis
```

8.6 Definition of Done

A user must be able to perform the following entirely through the browser:

```
Register
|
Login
|
Create Project
|
Upload CSV
|
Run Analysis
|
View Statistics
|
View Charts
```

9 Phase 6 – Document Intelligence and RAG

9.1 Objective

Implement document processing and Retrieval-Augmented Generation.

9.2 Tools

- LangChain
- Hugging Face
- Embedding Model
- Vector Store
- PDF Loader
- Text Splitter
- PostgreSQL + pgvector or selected vector database

9.3 Tasks

1. Implement document upload.
2. Validate documents.
3. Extract text.
4. Clean text.
5. Split documents into chunks.
6. Generate embeddings.
7. Store embeddings.
8. Implement similarity search.
9. Implement retriever.
10. Integrate LLM.
11. Implement RAG chain.
12. Return grounded answers.

9.4 RAG Pipeline

```
Document
|
v
Text Extraction
|
v
Text Chunking
|
v
Embeddings
|
```

```
    v
Vector Store
    |
    v
Retriever
    |
    v
Relevant Context
    |
    v
Hugging Face LLM
    |
    v
Answer
```

9.5 Definition of Done

A user must be able to:

1. Upload a technical document.
2. Ask a question about the document.
3. Retrieve relevant information.
4. Receive an AI-generated answer based on the retrieved context.

10 Phase 7 – GenAI and Data-Aware AI

10.1 Objective

Connect the AI system with both the document retrieval system and the data-analysis engine.

This phase transforms the application from a document chatbot into a data-aware engineering AI assistant.

10.2 Tools

- LangChain
- Hugging Face
- Pandas
- NumPy
- RAG
- LangChain Tools

10.3 AI Tools

Potential tools include:

```
DatasetSummaryTool
StatisticsTool
CorrelationTool
OutlierDetectionTool
DocumentSearchTool
```

10.4 AI Architecture

```
User Question
  |
  v
AI Assistant
  |
  v
LangChain
  |
  +-----+
  | |
  v v
Dataset Tools RAG Tools
  | |
  v v
Pandas / NumPy Document Retrieval
  | |
  +-----+-----+
                |
                v
            Hugging Face LLM
                |
                v
            Response
```

10.5 Example

User:

“Why did the temperature increase significantly during cycle 120?”

Potential system workflow:

1. Identify relevant dataset.
2. Retrieve cycle 120 data.
3. Calculate temperature statistics.
4. Analyze current and voltage.

5. Search technical documentation.
6. Combine retrieved information.
7. Generate final response.

10.6 Security Principle

The LLM should not receive unrestricted access to the operating system, database or arbitrary Python execution.

The architecture should expose controlled tools:

```
LLM
|
v
Approved Tool
|
v
Validated Operation
|
v
Result
```

10.7 Definition of Done

The AI assistant must be capable of answering questions using:

```
Structured Dataset
+
Data Analysis
+
Technical Documents
+
RAG
+
LLM
```

11 Phase 8 – Docker and Production Engineering

11.1 Objective

Containerize the entire application and create a reproducible development environment.

11.2 Tools

- Docker
- Docker Compose
- Nginx
- Environment Variables

11.3 Services

Initial services:

```
frontend
backend
postgres
```

Potential future services:

```
redis
worker
vector-db
```

These additional services should only be introduced when technically justified.

11.4 Docker Architecture

```
Docker Compose
  |
  +-----+
  | | |
  v v v
Frontend Backend PostgreSQL
React FastAPI
      |
      +-----+-----+
      | | |
      v v v
    Pandas LangChain HF
```

11.5 Tasks

1. Create frontend Dockerfile.
2. Create backend Dockerfile.
3. Configure PostgreSQL container.
4. Create Docker Compose file.
5. Configure environment variables.
6. Configure service networking.
7. Configure persistent database storage.
8. Configure logging.
9. Add Nginx if required.

11.6 Definition of Done

The application should be runnable using:

```
docker compose up
```

A new developer should be able to clone the repository and reproduce the development environment.

12 Phase 9 – Testing, Security and CI/CD

12.1 Objective

Improve reliability, security and development automation.

12.2 Tools

- Pytest
- Postman
- GitHub Actions
- Ruff
- Black
- ESLint
- Docker

12.3 Backend Testing

Test:

- Authentication
- Authorization
- Projects
- Datasets
- Analysis
- Documents
- Chat

12.4 Data Testing

Test:

- Statistical calculations
- Missing values
- Correlation
- Outlier detection
- Invalid datasets

12.5 Security Testing

Verify:

- Password hashing
- JWT validation
- Authorization
- Input validation
- File validation
- CORS configuration
- Secret management
- SQL injection protection

12.6 CI Pipeline

```
Git Push
|
v
GitHub
|
v
GitHub Actions
|
+---- Backend Tests
|
+---- Frontend Checks
|
+---- Linting
|
+---- Docker Build
|
v
Build Status
```

12.7 Definition of Done

Every major GitHub change should pass the automated checks before being merged into the main development branch.

13 Phase 10 – Deployment and Final Product Release

13.1 Objective

Deploy the completed application and prepare it as a professional portfolio project.

13.2 Tools

- Docker
- GitHub
- Cloud Platform
- PostgreSQL Hosting
- Nginx
- GitHub Actions

13.3 Tasks

1. Select deployment platform.
2. Configure production environment.
3. Deploy backend.
4. Deploy frontend.
5. Configure production database.
6. Configure HTTPS.
7. Configure CORS.
8. Configure environment variables.
9. Configure logging.
10. Configure health checks.
11. Perform production testing.

13.4 Final Documentation

The repository should contain:

```
README.md

docs/
|
+-- requirements.md
+-- architecture.md
+-- database.md
+-- api.md
+-- ai-architecture.md
+-- deployment.md
+-- development.md
```

13.5 Portfolio Materials

The final project should include:

- GitHub repository.
- Live application.
- Architecture diagram.
- Database ER diagram.
- API documentation.
- Screenshots.
- Demo video.
- Project documentation.
- Resume project description.

13.6 Definition of Done

The project is considered complete when:

- The application is deployed.
- Core features work in production.
- Authentication works.
- Dataset analysis works.
- Document RAG works.
- AI assistant works.

- Docker configuration works.
- Automated tests are available.
- CI pipeline is working.
- Documentation is complete.
- GitHub repository is organized.

14 Standard Development Workflow

Every phase should follow the same development cycle.

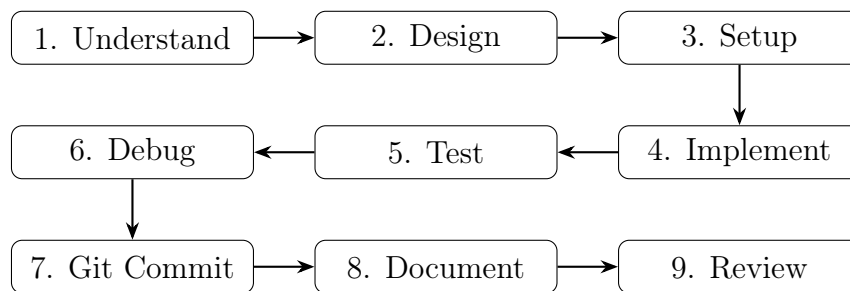


Figure 2: Standard Development Workflow

The workflow is:

1. Understand the requirement.
2. Design the solution.
3. Configure the required environment.
4. Implement the feature.
5. Test the feature.
6. Debug and improve.
7. Commit the completed work.
8. Document the implementation.
9. Review the phase.
10. Move to the next phase.

15 Git and GitHub Strategy

The project should use Git throughout development.

A possible branch structure is:

```
main
|
+-- develop
    |
    +-- feature/authentication
    +-- feature/projects
    +-- feature/dataset-analysis
    +-- feature/frontend
    +-- feature/rag
    +-- feature/ai-agent
    +-- feature/docker
    +-- feature/testing
```

Feature branches should be merged only after the feature has been tested.

16 Target Project Structure

The final repository is expected to approximately follow:

```
engisense-ai/
|
+-- frontend/
| |
| | +-- src/
| | +-- public/
| | +-- package.json
| | +-- Dockerfile
| |
+-- backend/
| |
| | +-- app/
| | |
| | | +-- api/
| | | +-- core/
| | | +-- database/
| | | +-- models/
| | | +-- schemas/
| | | +-- services/
| | | +-- analysis/
| | | +-- ai/
| | | +-- main.py
| | |
| | +-- tests/
| | +-- requirements.txt
| | +-- Dockerfile
| |
```

```
+-- docs/
|
+-- data/
|
+-- docker-compose.yml
+-- .env.example
+-- .gitignore
+-- README.md
+-- LICENSE
```

The exact directory structure may evolve as the architecture becomes more detailed.

17 Global Definition of Done

A feature is considered complete only when the following conditions are satisfied where applicable:

1. Requirement is clearly understood.
2. Implementation is complete.
3. Code follows the project structure.
4. Input validation is implemented.
5. Errors are handled appropriately.
6. Feature has been tested.
7. Existing functionality is not broken.
8. Git commit has been created.
9. Relevant documentation has been updated.

A phase is complete only when its specific Definition of Done has been satisfied.

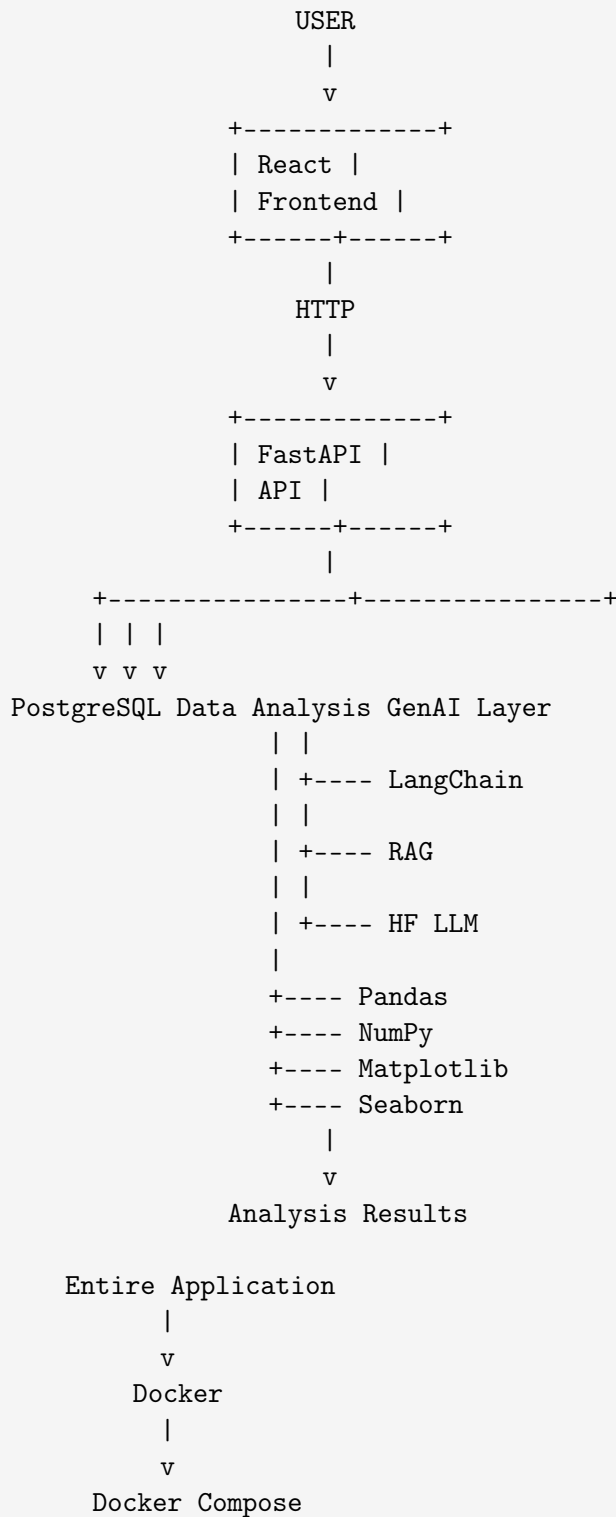
18 Master Progress Checklist

No.	Phase	Status	Completion Notes
1	Product Planning and Architecture	☐	
2	FastAPI Backend and PostgreSQL	☐	
3	REST APIs and Authentication	☐	
4	Engineering Data Analytics	☐	
5	React Frontend	☐	
6	Document Intelligence and RAG	☐	
7	GenAI and Data-Aware AI	☐	

- | | | |
|----|---------------------------------|---|
| 8 | Docker and Production Engineer- | □ |
| | ing | |
| 9 | Testing, Security and CI/CD | □ |
| 10 | Deployment and Final Release | □ |
-

19 Expected Final Architecture

The final system should conceptually follow:



|
v
Production Environment

20 Final Project Goal

The final objective is to develop a production-oriented full-stack AI application that demonstrates the integration of:

- Frontend engineering.
- Backend engineering.
- REST API development.
- Relational database management.
- Data analysis.
- Data visualization.
- Document processing.
- Retrieval-Augmented Generation.
- Open-source LLM integration.
- AI tool orchestration.
- Containerization.
- Automated testing.
- CI/CD.
- Deployment.

The project should be developed incrementally, with each phase producing a working and testable component.

The final system should not simply demonstrate individual technologies. It should demonstrate the ability to design, implement, integrate, test, document and deploy a complete software product.

END OF DEVELOPMENT ROADMAP